Cloud-Native Movie Ticket Booking System Using Java Microservices

1. Introduction

The Cloud-Native Movie Ticket Booking System is a modern distributed web application developed using Java Spring Boot Microservices architecture to provide a secure, scalable, and efficient platform for online movie ticket booking. The system allows users to register and log in securely, search for movies based on city, genre, language, or date, view theatre and show details, select seats, make online payments, receive QR code-based electronic tickets, and manage their booking history through a user-friendly interface.

2. Problem Statement

The rapid growth of online movie ticket booking has significantly increased the demand for reliable, scalable, and user-friendly booking platforms. However, many existing movie ticket booking systems are developed using traditional monolithic architectures, where all application components are tightly coupled into a single system. As the number of users and transactions increases, these systems become difficult to scale, maintain, and update. Even a minor modification or deployment often requires the entire application to be rebuilt and redeployed, resulting in increased downtime and maintenance efforts.

During peak booking periods, such as weekends, holidays, and the release of highly anticipated movies, monolithic systems frequently experience performance degradation, slow response times, and server overload. One of the most common issues is double booking, where multiple users attempt to reserve the same seat simultaneously due to the absence of an effective real-time seat locking mechanism. Additionally, many existing systems provide limited support for dynamic pricing, automated notifications, digital ticket generation, loyalty programs, and efficient refund processing, leading to a poor user experience and increased operational challenges for theatre administrators.

Another major limitation of traditional systems is their lack of cloud-native capabilities. They often cannot support independent service scaling, fault tolerance, distributed data management, or automated deployment. A failure in one module can affect the entire application, reducing system availability and reliability. Furthermore, integrating new features or third-party services becomes increasingly complex as the application grows.

To address these challenges, there is a need for a modern movie ticket booking platform based on microservices architecture. The proposed Cloud-Native Movie Ticket Booking System divides business functionalities into independent services, each with its own database, enabling better scalability, maintainability, and fault isolation. By leveraging Spring Boot Microservices, Apache Kafka, Docker, Kubernetes, and AWS Cloud Services, the system provides real-time seat management, secure payment processing, event-driven communication, automated deployment, and high availability, delivering a seamless and efficient booking experience for both customers and theatre administrators.

3. Objectives

The primary objectives of this project are:

- Design and develop a scalable movie ticket booking platform using Java Microservices.
- Implement secure user authentication using JWT and role-based authorization.
- Provide efficient movie, theatre, screen, and show management.
- Enable real-time seat locking to prevent double booking.
- Implement secure online payment and refund processing.
- Generate QR code-based electronic tickets.
- Provide automated email/SMS notifications.
- Implement dynamic ticket pricing based on demand and seat category.
- Develop review and rating modules for movies and theatres.
- Build an analytics dashboard for administrators.
- Deploy the application using Docker and Kubernetes.
- Integrate AWS cloud services for storage and database management.
- Implement centralized logging, monitoring, and fault tolerance.
-

4. Scope of the Project

The project covers the complete lifecycle of online movie ticket booking.

Functional Scope

- User registration and login
- Role-based access control
- Movie management
- Theatre and screen management
- Show scheduling
- Movie search and filtering
- Seat selection
- Seat locking
- Ticket booking
- Online payment
- Booking cancellation
- Refund processing
- QR code ticket generation
- Email and SMS notifications
- Review and rating management
- Loyalty rewards
- Coupons and promotional offers
- Waitlist management
- Booking history
- Analytics and reporting

Technical Scope

- Java Spring Boot Microservices
- Spring Cloud components
- API Gateway
- Eureka Service Discovery
- Config Server
- OpenFeign
- Resilience4j
- Apache Kafka Event Streaming
- REST APIs
- JWT Security
- Docker
- Kubernetes
- AWS Cloud Deployment
- CI/CD Pipeline
- Prometheus and Grafana Monitoring
- ELK Stack Logging

5. Technologies Used

Backend

- Java 21 (or Java 17 LTS)
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- Maven

Microservices

- Spring Cloud Gateway
- Eureka Server
- Config Server
- OpenFeign
- Resilience4j

Messaging

- Apache Kafka

Database

- PostgreSQL / MySQL
- Separate database for each microservice

Security

- JWT Authentication
- BCrypt Password Encryption
- Role-Based Authorization

Cloud & DevOps

- Docker
- Docker Compose
- Kubernetes (Minikube or AWS EKS)
- AWS EC2
- AWS RDS
- AWS S3
- GitHub Actions or Jenkins

Monitoring

- Prometheus
- Grafana
- ELK Stack

Documentation & Testing

- Swagger/OpenAPI
- Postman
- JUnit
- Mockito

Frontend

- React.js
- CSS
- Bootstrap
- JavaScript

6. Expected Outcome

The expected outcome of this project is a fully functional, scalable, and cloud-native movie ticket booking platform capable of handling large numbers of concurrent users with high availability and reliability.

The system will:

- Provide secure authentication and authorization.
- Enable users to search and book movie tickets efficiently.
- Prevent double booking through real-time seat locking.
- Support secure online payment and automated refunds.
- Generate QR code-based electronic tickets.
- Deliver instant booking and cancellation notifications.
- Implement dynamic pricing and promotional offers.
- Maintain separate databases for each microservice.
- Ensure fault tolerance using Spring Cloud and Resilience4j.
- Support asynchronous communication using Apache Kafka.
- Be deployable on cloud platforms using Docker and Kubernetes.
- Offer centralized monitoring, logging, and analytics for administrators.

Overall, the project demonstrates modern software engineering practices, cloud-native architecture, distributed system design, DevOps integration, and scalable microservices development, making it suitable as a major academic project and a strong portfolio project for industry roles.